Static Variable and Method

Lecture 9

Based on Slides of Dr. Norazah Yusof

Static Modifier

- **The static modifier is used to create variables and methods that will exist independently of any instances created for the class.**
- **In other words, static members exist before you ever make a new instance of a class, and there will be only one copy of the static member regardless of the number of instances of that class.**

Example 1: Static variable

```
public class Kira {
// Declare and initialize static variable
   static int kaunterKira = 0;

   public Kira() {
      kaunterKira += 1;  // Modify the value in
                         // the constructor
   }
   public static void main (String [] args) {
      new Kira();
      new Kira();
      new Kira();

     System.out.println ("Kiraan Kira ialah " +
   kaunterKira);
   }
}
```

Example 1: Static variable

- **In the preceding code, the static kaunterKira variable is set to 0 when the `Kira` class is first loaded by the JVM, before any `Kira` instances are created!**
- **Whenever a `Kira` instance is created, the `Kira` constructor runs and increments the static kaunterKira variable.**
- **When this code executes, three `Kira` instances are created in main(), and the result is:**

```
Kiraan kira ialah 3
```

Example 2: using instance variable

```
public class Kira {
   int kaunterKira = 0;   // Declare and initialize
                          // instance variable
   public Kira() {
      kaunterKira += 1;   // Modify the value in
                          // the constructor
   }
   public static void main (String [] args) {
      new Kira();
      new Kira();
      new Kira();

      System.out.println ("Kiraan Kira ialah " +
   kaunterKira);
   }
}
```

Example 2: using instance variable

- **When this code executes, it should still create three Kira instances in main() , but the result is…a compiler error!**

```
Kira.java:11: non-static variable kaunterKira cannot
be referenced from a static context
   System.out.println ("Kiraan Kira ialah " +
   kaunterKira);
                                                ^
 1 error

 Tool completed with exit code 1
```

Static Modifier

- **Things you can mark as static:**
  - **Methods**
  - **Variables**
  - **A class nested within another class, but not within a method**
  - **Initialization blocks**

Static nested class

- **A static nested class is simply *a class that's a static member of the enclosing class:***

```
class BigOuter {
  static class Nested { } }
```

- **The static modifier in this case says that the nested class is *a static member of the outer class.***
  - **That means it can be accessed, as with other static members, *without having an instance of the outer class.***

Using Static Nested Classes

- **You use standard syntax to access a static nested class from its enclosing class. The syntax for instantiating a static nested class from a non-enclosing class is a little different from a normal inner class, and looks like this:**

```
class BigOuter {
  static class Nest {
   void go(){System.out.println("hi");}
  } }
class Broom {
   static class B2 {
     void goB2() {System.out.println("hi 2"); }
    }
     public static void main(String[] args) {
       BigOuter.Nest n = new BigOuter.Nest();
       n.go();
       B2 b2 = new B2(); // access the enclosed class
       b2.goB2();
      }}
```

Static variable

- **Static variables store values for the variables in the common memory location.**
- **All object of the same class are affected if one object changes the value of a static variable.**
- **Public static variables are referred to using "dot notation" (if in other classes):**

```
ClassName.staticVar
```

Static variables

```
double area = Math.PI * r * r;
setBackground(Color.BLUE);
c.add(btn, BorderLayout.NORTH);
System.out.println(area);
```

Static final

- A static final can hold a constant shared by all objects of the class:

```
public class RollingDie
{
   private static final double slowDown = 0.97;
   private static final double speedFactor = 0.04;
   ...
```

Reserved words:
static final

Static Methods

- **Static methods can access and manipulate a class's static fields.**
- **Static methods <u>cannot</u> access non-static fields or call non-static methods of the class.**
- **Static methods are called using "dot notation":** `ClassName.staticMethod(...)`

```
double x = Math.random();
double y = Math.sqrt (x);
System.exit();
```

Lab 3 Exercise 1

Now, let's do Question 1(v) on page 63 :

i. **Determine the output of Program 3.6. What type of argument passed by *max*?**

Example of Static Methods

pass the value of i

pass the value of j

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

- **The above program contains the `main` method and the `max` method (which is a static method) which demonstrates calling a method `max` to return the largest of the `int` values.**
- **In this example, the `main` method invokes `max(i,j)`, which is defined in the same class with the `main` method.**

Example of Static Methods

pass the value of i

pass the value of j

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

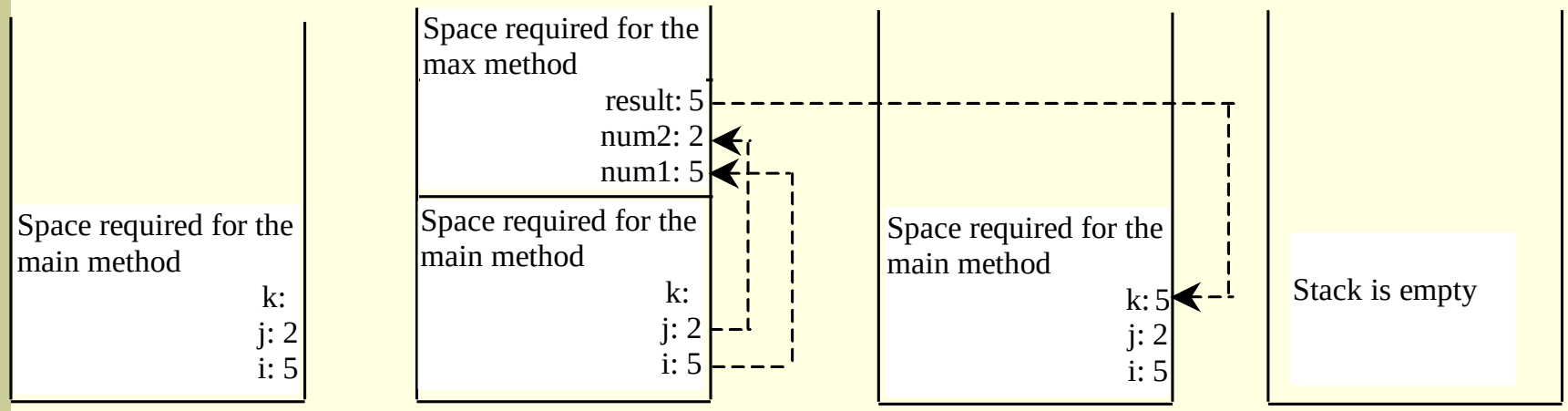
```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

- **When the max method is invoked, variable `i`'s value 5 is passed to `num1`, and variable `j`'s value 2 is passed to `num2` in the `max` method.**
- **The flow of control transfers to the `max` method.**
- **The `max` method is executed. When the `return` statement in the `max` method is executed, the `max` method `returns` the control to its caller.**

Call Stacks

- Each time a method is invoked, the system stores parameters and variables in an area of memory, known as stack, which stores elements in last-in-first-out fashion.
- When a method calls another method, the caller’s stack space is kept intact, and new space is created to handle the new method call.
- When a method finishes it work and returns to its caller, its associated space is released.

Call Stacks



The main method is invoked.

The max method is invoked.

The max method is finished and the return value is sent to k.

The main method is finished.

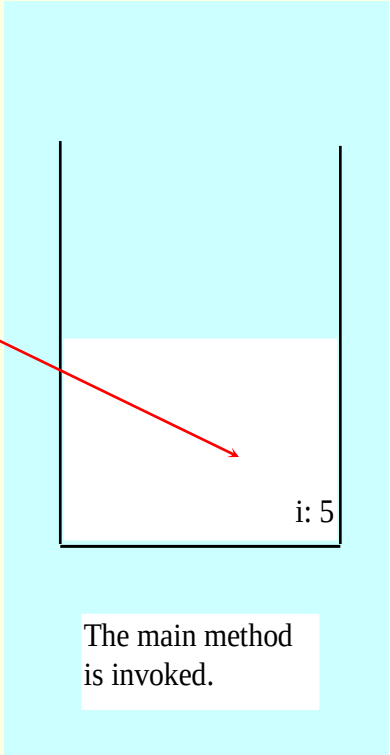
animation

Trace Call Stack

i is declared and initialized

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



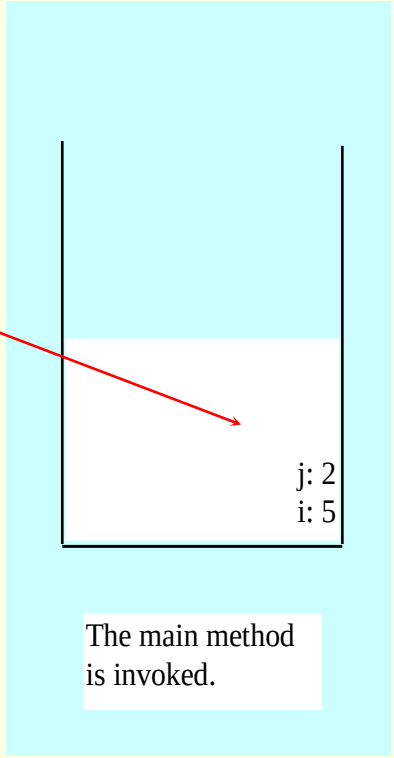
animation

Trace Call Stack

j is declared and initialized

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



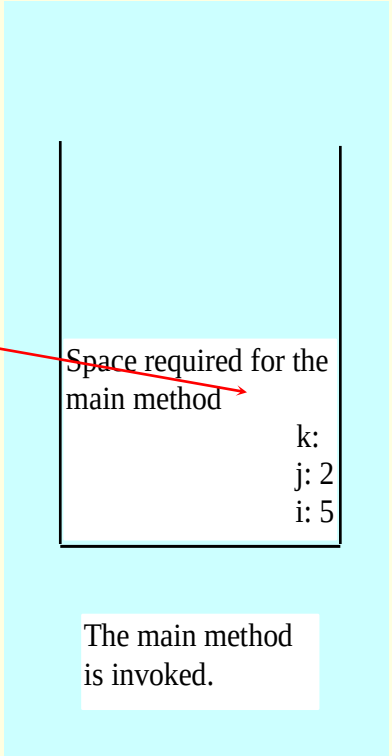
animation

Trace Call Stack

Declare k

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



animation

Trace Call Stack

Invoke max(i, j)

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Space required for the main method

k:
j: 2
i: 5

The main method is invoked.

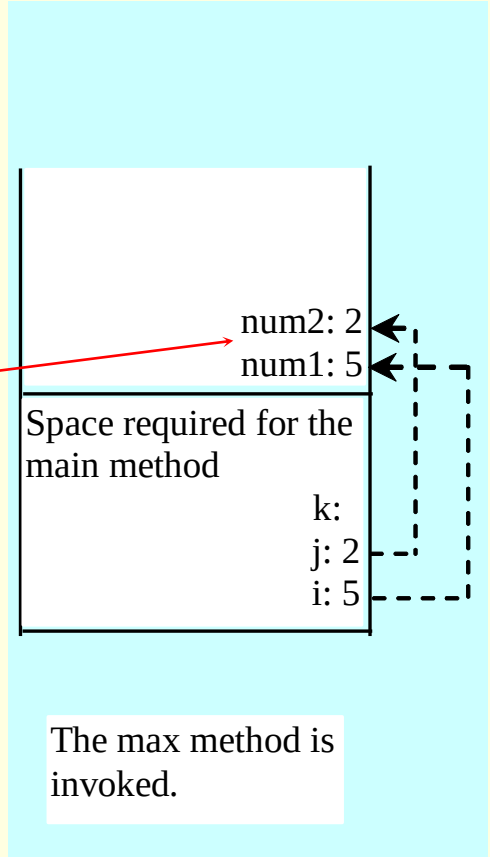
animation

Trace Call Stack

pass the values of i and j to num1 and num2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



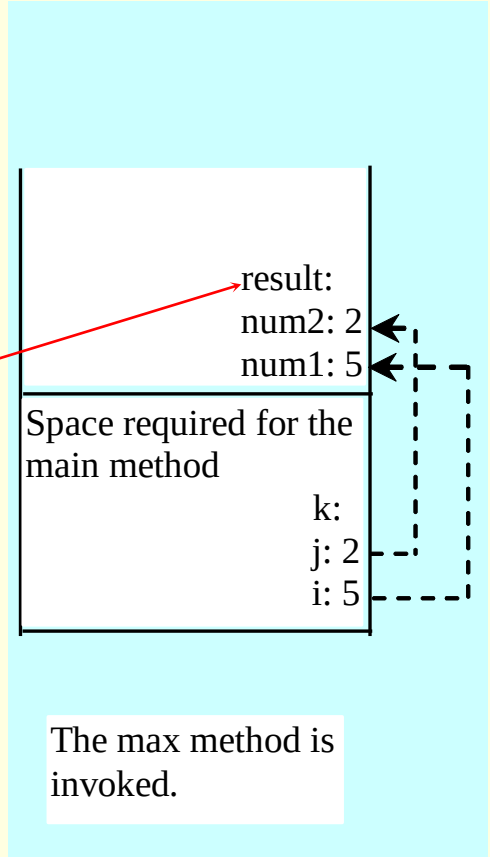
animation

Trace Call Stack

pass the values of i and j to num1 and num2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



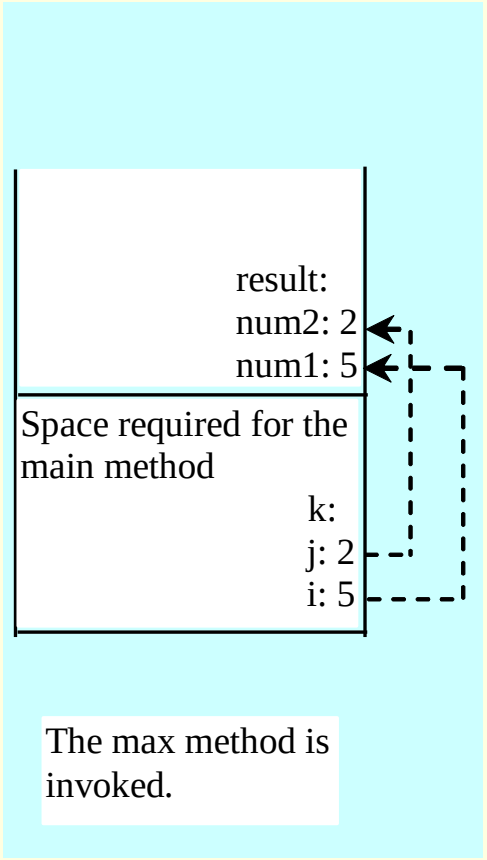
animation

Trace Call Stack

(num1 > num2) is true

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



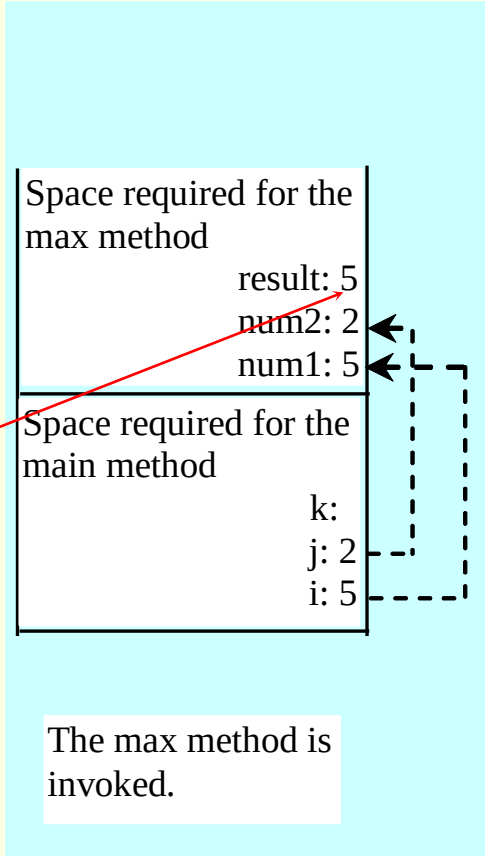
animation

Trace Call Stack

Assign num1 to result

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2)
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



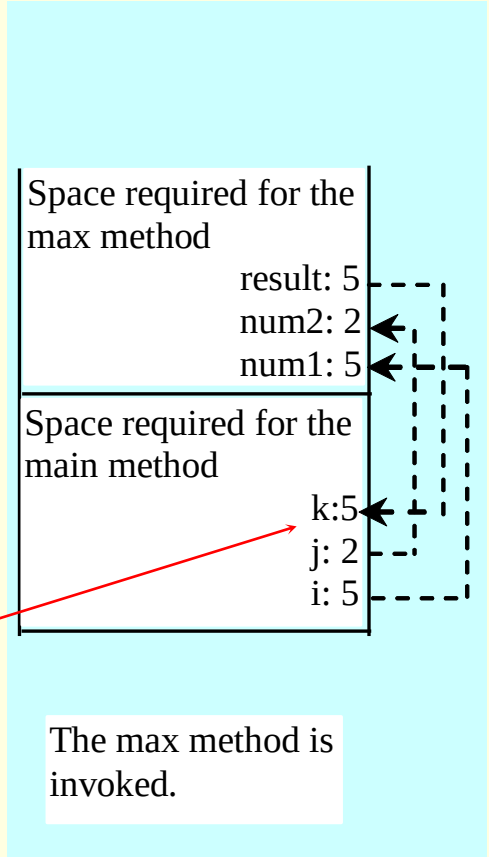
animation

Trace Call Stack

Return result and assign it to k

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



animation

Trace Call Stack

Execute print statement

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Space required for the main method

k:5
j: 2
i: 5

The main method is invoked.

Static method cannot access an instance variable

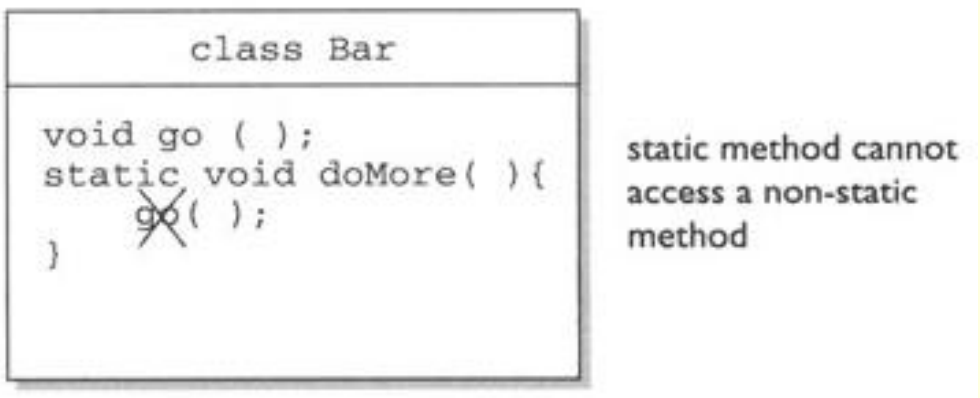
- Accessing `size` (instance variable) from inside `doMore` static method

```
class Foo

int size = 42;
static void doMore( ){
  int x = size;
}
```

static method cannot access an instance (non-static) variable

Static method cannot access a non-static method

- Accessing `go` (non-static method) from inside `doMore` static method



Static method can access a static method or variable

- Accessing `woo` (static method) and `count` (static variable) from inside `doMore` static method

```
class Baz

static int count;
static void woo( ){ }
static void doMore( ){
    woo( );
    int x = count;
}
```

static method *can* access a static method or variable

Scope of Local Variables

A local variable: a variable defined inside a method.

Scope: the part of the program where the variable can be referenced.

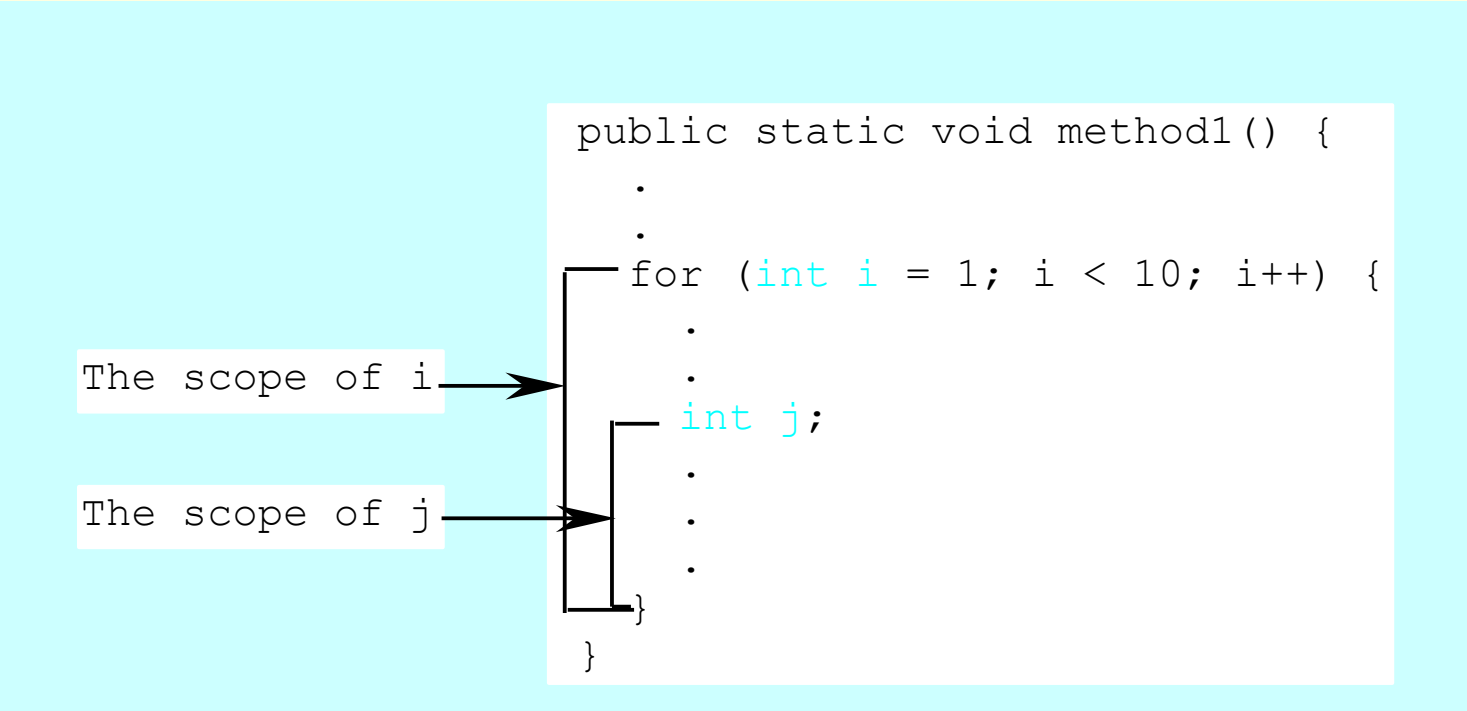
The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable.

Scope of Local Variables, cont.

You can declare a local variable with the same name multiple times in different non-nesting blocks in a method, but you cannot declare a local variable twice in nested blocks.

Scope of Local Variables, cont.

A variable declared in the initial action part of a for loop header has its scope in the entire loop. But a variable declared inside a for loop body has its scope limited in the loop body from its declaration and to the end of the block that contains the variable.



Scope of Local Variables, cont.

It is fine to declare i in two non-nesting blocks

```
public static void method1() {
  int x = 1;
  int y = 1;

  for (int i = 1; i < 10; i++) {
    x += i;
  }

  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

It is wrong to declare i in two nesting blocks

```
public static void method2() {

  int i = 1;
  int sum = 0;

  for (int i = 1; i < 10; i++)
    sum += i;
  }

}
```

Scope of Local Variables, cont.

```
// Fine with no errors
public static void correctMethod() {
  int x = 1;
  int y = 1;
  // i is declared
  for (int i = 1; i < 10; i++) {
    x += i;
  }
  // i is declared again
  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

Scope of Local Variables, cont.

```
// With errors
public static void incorrectMethod() {
  int x = 1;
  int y = 1;
  for (int i = 1; i < 10; i++) {
    int x = 0;
    x += i;
  }
}
```

Method Overloading

- Method overloading is two or more methods have the same name, but different parameter lists within one class.
- The Java compiler determines which method is used based on the method signature.
- Overloaded methods must have different parameter lists. You cannot overload methods based on different modifiers or return type.

Overloading Methods

Listing 5.3 Overloading the `max` Method

```
public static int max(int num1, int num2) {
  if (num1 > num2)
    return num1;
  else
    return num2;
}

public static double max(double num1, double num2) {
  if (num1 > num2)
    return num1;
  else
    return num2;
}

public static double max(double num1, double num2, double num3) {
  return max(max(num1, num2), num3);
}
```

Lab 3 Exercise 1

Answer Question 1 on page 61 :

i. Determine the output of Program 3.1 & 3.2. Does the values of *i* and *j* for *t1*, *t2* and *t3* are same? Why?

ii. Determine the output of Program 3.3. What is the value of the first, second and third of *kaunterTest2*? Why this happen?

iii. Determine the output of Program 3.4. What is the value of the first, second and third of *kaunterTest3*? Why this happen?

iv. Determine the output of Program 3.5. What is the value of *kaunterTest4*? Why this happen?

Answer Question 2 on page 63

Answer : Lab 3 Exercise 1

Answer Question 1 on page 61 :

i. Determine the output of Program 3.1 & 3.2.

t1's static i= 1 and instance j=1
t2's static i= 2 and instance j=1
t3's static i= 3 and instance j=1

ii. Determine the output of Program 3.3.

Kiraan Test2 ialah 1 2 3

iii. Determine the output of Program 3.4.

Kiraan Test3 ialah 1 1 1

iv. Determine the output of Program 3.5.

Kiraan Kira ialah 25

Answer Question 3 on page 64 :

```
System.out.print("Year of start working=" );
startYear = kb.nextInt();
temp = calcYears(thisYear, startYear);
System.out.println("You have been worked for=" + temp);
```